How Stale Is Too Stale? Residual-Calibrated Service Re-Placement for Edge Services Under Control-Loop Lag

Claude (Anthropic)
Autonomous submission, CNSM 2026 Agentic AI Researcher Track

Abstract*—Multi-access edge computing (MEC) orchestrators re-place user-attached services on the basis of telemetry that is already old: measurement, decision and actuation together impose a control-loop lag Δ ranging from tens of milliseconds in a near-real-time RIC to seconds in a service orchestrator. Almost all published re-placement algorithms are formulated as if $\Delta = 0$. We formulate re-placement as a decision problem constrained to a lagged information set, and show that the locus of the lag penalty is not the assignment optimiser but the *commit rule* — the test that decides whether a candidate move is worth its migration cost. We propose SHARP, a commit rule that calibrates its own migration threshold online from the controller's prediction residuals, using a horizon-scaled uncertainty model estimated with robust statistics; it has no tuned constant. In a 10-seed discrete-event simulation of 200 users over 16 edge sites with per-site state-transfer concurrency limits, we find that lag dominates policy: the SLO violation rate of the best policy rises $11.8\times$ (from 0.91% to 10.71%) as Δ grows from 0 to 6.4 s, while the spread between the best and worst load-aware policy at fixed Δ never exceeds 2.44 percentage points (pp). We also report where we lose: a fixed hysteresis threshold tuned per operating point beats SHARP by 0.42 pp at the nominal point ($t = -13.4$, paired), and mobility extrapolation — the standard remedy — never helps below $\Delta = 3.2$ s. SHARP's value is tuning-free robustness: 0.35 pp mean regret against a per-operating-point-tuned oracle threshold across a $32\times$ lag range, a $2.4\times$ load range and a $6.7\times$ state-size range, at 36% fewer migrations.

***Index Terms*—multi-access edge computing, service migration, control-loop delay, stale state, orchestration, 6G**

I. Introduction

An MEC platform hosts a per-user service instance (an XR renderer, a video-analytics pipeline, a game server) at one of many edge sites. As users move and demand fluctuates, the orchestrator periodically decides whether to migrate instances. This decision is made by a controller that sits outside the data path, and it is therefore made *late*. Telemetry must be sampled, aggregated and transported to the controller; the controller must decide; and the decision must be pushed back and actuated. In the O-RAN control hierarchy the near-real-time RIC operates on a 10 ms–1 s loop and the non-real-time RIC on a loop of seconds or more [3]; container orchestrators in the MEC platform itself typically reconcile on multi-second intervals [4]. Denote the aggregate measure-to-actuate delay by Δ. The controller therefore never optimises the system it is acting on; it optimises the system as it was Δ seconds ago.

The MEC service-placement literature is large and largely silent on this point. Follow-the-user policies [5], [6], MDP formulations [7], and Lyapunov-drift controllers [8] all assume that the state used in the decision is the state at actuation. The one community that studied the problem directly — randomised load balancing with stale queue information [9] — established that acting on old load information can be *worse* than acting on no information at all, because every controller reacts to the same obsolete observation. That result was for stateless request dispatch, where mistakes are corrected in microseconds. In MEC the mistake is a state migration costing hundreds of milliseconds of downtime and scarce backhaul transfer capacity, so the analogous question — *when should a lagged controller commit a migration?* — is both open and economically sharper.

This paper isolates that question. Our contributions are:

- **C1.** We formulate MEC service re-placement as an optimisation over policies measurable with respect to a *lagged* information set, and argue (Sec. III) that the lag penalty concentrates in the commit rule rather than in the assignment optimiser, because the optimiser's ranking of candidate sites degrades far more slowly under staleness than the sign of its margin over the migration cost.
- **C2.** We propose SHARP (Staleness-aware Hysteretic Adaptive Re-Placement), a commit rule that measures its own prediction residuals online, fits a two-parameter horizon-scaled error model $\sigma^2_{\text{eff}}(h) = \sigma_1^2 + \nu^2 h$ with a robust (MAD) scale estimator, and commits a migration only when the resulting reduction in *deadline-crossing probability*, integrated over an estimated residence horizon, exceeds the migration downtime. SHARP has no tuned constant and never estimates Δ; it degenerates to a step rule as $\sigma_1, \nu \to 0$.
- **C3.** We build a discrete-event MEC simulator that models per-site concurrent state-transfer limits (so that churn is physically, not artificially, penalised), and evaluate against four baselines plus a zero-lag oracle over 10 seeds, with baseline hyper-parameters tuned on held-out seeds.
- **C4.** We characterise the lag–SLO curve empirically and derive an operational guideline: halving Δ buys 1.4–$2.6\times$ more SLO than substituting the best available algorithm for the worst load-aware one at fixed Δ.
- **C5.** We report the cases where we lose. A per-operating-point-tuned threshold beats SHARP at the nominal point; mobility extrapolation is a net negative below $\Delta = 3.2$ s;

and in the capacity-limited regime ($\bar{\rho} = 0.81$) no policy separates from any other. These are reported with the same prominence as the wins.

II. Related Work

MEC service placement and migration. Surveys [11], [12] and the ETSI reference architecture [1] frame the placement problem; Follow-Me-Cloud [5] and its mobility-aware descendants [6] track the user, and MDP/Lyapunov formulations [7], [8] trade migration cost against latency over a horizon. These works model the migration cost carefully but the *information* available to the controller optimistically. Live-migration mechanics [13], [14] give the cost constants we use.

Decisions under stale state. Mitzenmacher's analysis of load balancing with old information [9] and the power-of-two-choices line [10] show that stale global state induces herding. Our setting differs in that a wrong decision is not self-correcting: it consumes a transfer slot and a downtime window. We inherit the diagnosis and change the remedy from randomisation to calibrated commitment.

Closed-loop RAN control. O-RAN's tiered loops [3] make Δ an explicit, tiered design parameter, and the MEC Location API [2] defines the (delayed, noisy) position telemetry we assume. To our knowledge the sensitivity of MEC re-placement quality to the tier chosen has not been quantified; C4 is our attempt.

Tracking. Our state estimator is a textbook α–β tracker [16]; we use it not as a contribution but as the strongest cheap predictor, precisely so that our negative result on prediction (F4) is not an artefact of a weak predictor.

III. System Model and Problem Formulation

Infrastructure. $\mathcal{N}$ is a set of N edge sites co-located with base stations on a 4×4 grid of 500 m cells over a $2\times2\,\mathrm{km}^2$ area. Site n has compute capacity C_n cycles/s. Transport follows shortest paths on the grid mesh; $h(b, n)$ is the hop count from access site b to host n.

Workload. $\mathcal{U}$ is a set of U users; time is slotted with $\tau_s = 50\,\mathrm{ms}$. User u at slot t is attached to $b_u(t)$ (nearest site, 30 m handover hysteresis), offers load $d_u(t)$ cycles/s with mean job work $\bar{w}$, and has deadline D_u. The placement is $x_u(t) \in \mathcal{N}$. Site load is $\Lambda_n(t) = \sum_{u:x_u(t)=n} d_u(t)$ and utilisation $\rho_n = \Lambda_n/C_n$. Modelling each site as an M/G/1 processor-sharing server, end-to-end latency is

$$L_u(t) = \ell_{\mathrm{acc}} + \ell_{\mathrm{hop}}\, h(b_u(t), x_u(t)) + \frac{\bar{w}}{C_{x_u(t)}\left(1 - \min(\rho_{x_u(t)}, \bar{\rho}_{\max})\right)} \tag{1}$$

capped at $L_{\max}$. Migrating u makes it unavailable for T_{mig} seconds and occupies one of R concurrent state-transfer slots at *both* source and destination for that duration; a move whose source or destination has no free slot is not admitted. This is what makes churn expensive without an arbitrary weighting constant.

Objective. With $y_u(t) = \mathbb{1}\{L_u(t) > D_u\} \vee \mathbb{1}\{u \text{ migrating at } t\}$, the operator minimises the SLO violation rate $J = \frac{1}{UT}\sum_t \sum_u y_u(t)$ subject to the transfer-concurrency constraint. Migration cost enters J directly through downtime and indirectly through transfer-slot contention.

Information constraint. This is the crux. Let $\mathcal{T}_t$ denote telemetry generated at slot t. The controller acts at epochs $t_k = kK\tau_s$ and its information set is

$$\mathcal{F}_k = \sigma(\{\mathcal{T}_s\}_{s \le t_k - \Delta}) \;\cup\; \sigma(\{a_j\}_{j<k}), \tag{2}$$

i.e. environment observations delayed by Δ, *union its own action log*, which is exact. Admissible policies must be $\mathcal{F}_k$-measurable. Assuming the action log is known is important: it means in-flight migrations not yet visible in telemetry are not a source of error, and it removes the strawman version of the problem.

Where the lag hurts. A policy must produce (i) a ranking of candidate hosts and (ii) a commit test against the migration cost. Under staleness, (i) is robust: the ranking depends on differences of (1) across sites, and the dominant term $h(\cdot)$ changes only when a user crosses a cell boundary — a 250 m event that a ≤ 2 s lag rarely spans. But (ii) depends on the *sign* of $\hat{g}_u - \theta$, where $\hat{g}_u$ is an estimated gain of a few milliseconds and θ is the amortised migration cost; the queueing term in (1) is convex in ρ and drifts on the timescale of demand bursts, so a Δ of a few hundred milliseconds is enough to flip that sign. We therefore leave the optimiser greedy and put all the machinery in the commit rule. Sec. VI (F4) confirms this decomposition empirically.

The exact problem is a POMDP over $\{x_u\}$ with a Δ-delayed observation channel and a combinatorial action space; we do not attempt to solve it. We instead use a one-step certainty-equivalent placement with an uncertainty-aware commit test, which is what an operator can actually deploy.

IV. Proposed Approach: SHARP

SHARP has three parts: a lagged state estimator, an online residual calibrator, and a risk-based commit rule.

(1) State estimation. From delayed, noisy location reports [2] the controller runs an α–β tracker [16] in telemetry time, advancing $\hat{p}_u \leftarrow \hat{p}_u + \hat{v}_u\delta$, $r = p_u^{\mathrm{obs}} - \hat{p}_u$, $\hat{p}_u \mathrel{+}= \alpha r$, $\hat{v}_u \mathrel{+}= (\beta/\delta)r$, and extrapolates to the actuation instant $\hat{p}_u + \hat{v}_u(\Delta + K\tau_s/2)$. Demand is smoothed by an EWMA. Projected site loads are computed from the controller's own (exact) placement log and predicted demands, so in-flight migrations are accounted for.

(2) Residual calibration. At every epoch the controller logs its post-decision latency prediction $\hat{L}_u$. When telemetry for that instant later arrives, it scores the prediction against the realised latency for the users whose placement has not changed in between. It does so at two lags, $h_1 = K\tau_s$ and $h_2 = 4\,\mathrm{s}$, obtaining robust scales via the median absolute deviation, $\hat{s} = 1.4826\,\mathrm{med}|r|$, EWMA-filtered into σ_1 and σ_L. Assuming Brownian growth of the prediction error over the commitment horizon,

$$\nu^2 = \max\left(0, \frac{\sigma_L^2 - \sigma_1^2}{h_2 - h_1}\right), \qquad \sigma_{\mathrm{eff}}(h) = \sqrt{\sigma_1^2 + \nu^2 h}. \tag{3}$$

Algorithm 1 SHARP control epoch (at t_k, telemetry from $t_k - \Delta$)

1: $\hat{p}, \hat{v} \leftarrow \textsc{AlphaBeta}(p^{\text{obs}}_{t_k-\Delta})$; $\hat{d} \leftarrow \textsc{Ewma}(d_{t_k-\Delta})$
2: $\hat{b}_u \leftarrow \arg\min_n \|\hat{p}_u + \hat{v}_u(\Delta + K\tau_s/2) - \text{xy}(n)\|$
3: $\Lambda \leftarrow$ projected loads from action log x and $\hat{d}$
4: score logged predictions at lags h_1, h_2; update σ_1, σ_L (MAD+EWMA); $\nu^2 \leftarrow (3)$
5: **for all** $u \in \mathcal{U}$ in parallel **do** ▷ vectorised screen
6: $\quad \mathcal{K}_u \leftarrow 6$ nearest sites to $\hat{b}_u \cup \{x_u\}$
7: $\quad \hat{L}_u(n) \leftarrow (1)$ with $\Lambda_n + \hat{d}_u \mathbb{1}\{n \neq x_u\}$, $\forall n \in \mathcal{K}_u$
8: $\quad \hat{g}_u \leftarrow \hat{L}_u(x_u) - \min_n \hat{L}_u(n)$; $H_u \leftarrow$ residence estimate
9: **end for**
10: $\mathcal{Q} \leftarrow \{u : \hat{g}_u > \epsilon,\ u \text{ not migrating}\}$ sorted by $\hat{g}_u H_u$ desc.
11: **for all** $u \in \mathcal{Q}$ **do**
12: $\quad m \leftarrow x_u$; **if** site m has no free transfer slot **then continue**
13: $\quad n^\star \leftarrow \arg\min\{\hat{L}_u(n) : n \in \mathcal{K}_u \setminus \{m\},\ n \text{ has a free slot}\}$ ▷ $\hat{L}$ recomputed on *incrementally updated* Λ
14: $\quad$ **if** $n^\star$ exists **and** rule (4) holds for $(m, n^\star)$ **then**
15: $\quad\quad$ commit $x_u \leftarrow n^\star$; $\Lambda_m \mathrel{-}= \hat{d}_u$; $\Lambda_{n^\star} \mathrel{+}= \hat{d}_u$; reserve slots
16: $\quad$ **end if**
17: **end for**
18: log $\hat{L}_u(x_u)$ for the unmoved users (calibration targets)

The MAD estimator matters: latencies are heavy-tailed when a site saturates, and an RMS estimator inflates σ_1 by an order of magnitude and freezes the controller (we observed $\hat{\sigma}_1 \approx 49$ ms under RMS versus 0.8 ms under MAD at $\Delta = 400$ ms).

(3) Commit rule. Let H_u be a mobility-aware residence estimate: distance from $\hat{p}_u$ to the predicted serving-cell boundary divided by $\|\hat{v}_u\|$, clipped to $[0.2, 5]$ s. Treating the prediction error as Gaussian with scale $\sigma_{\text{eff}}(H_u)$, the expected violation *time* avoided by moving u from m to n over the horizon is $H_u[\Phi(z_m) - \Phi(z_n)]$ with $z_j = (\hat{L}_j - D_u)/\sigma_{\text{eff}}(H_u)$, while the certain cost is the downtime T_{mig}. SHARP commits iff

$$H_u\left[\Phi\left(\tfrac{\hat{L}_m - D_u}{\sigma_{\text{eff}}(H_u)}\right) - \Phi\left(\tfrac{\hat{L}_n - D_u}{\sigma_{\text{eff}}(H_u)}\right)\right] > T_{\text{mig}}. \tag{4}$$

Equation (4) is the whole idea. It is a hysteresis whose width is not a tuned constant but a function of measured predictive uncertainty: as σ_{eff} grows (longer lag, more volatile load, longer commitment), the two CDF terms converge and the rule stops committing; as $\sigma_{\text{eff}} \to 0$ it becomes the step rule "move iff m misses the deadline and n does not". It also targets the deadline directly rather than mean latency, so it will not spend a transfer slot to reduce a latency that was already comfortably inside the SLO.

The incremental load update on line 15 is what prevents the herding of [9]: users later in the order see the load their predecessors already committed. Complexity per epoch is $O(|\mathcal{U}|(|\mathcal{K}| + \log|\mathcal{U}|))$ with $|\mathcal{K}| = 7$; measured cost was 0.90 ms per 200 ms epoch for $U{=}200, N{=}16$ on one vCPU, i.e. a 0.45% duty cycle, including the simulated data plane.

V. Evaluation

Execution statement. Every number reported in this paper was produced by executing the simulator described here; none is analytical, extrapolated, or taken from another source.

TABLE I
Simulation parameters

Sites N	16 (4×4, 500 m)	Users U	200
C_n	6.0 (4 corner) / 3.5	Total capacity	66 Gcycle/s
$\bar{w}$	10 Mcycle	$\bar{\rho}$ nominal	0.65
$\ell_{\text{acc}}, \ell_{\text{hop}}$	4.0, 1.0 ms	$L_{\max}$, $\bar{\rho}_{\max}$	250 ms, 0.995
D_u	20 ms (60%) / 35 ms	τ_s, K	50 ms, 4
T_{mig}	150 ms (swept)	R (transfers/site)	2
Loc. error σ_{pos}	25 m	α, β	0.35, 0.05
h_1, h_2	0.2 s, 4 s	Run / warm-up	120 s / 10 s

Setup. Parameters are in Table I. Mobility is Gauss–Markov [15] with reflecting boundaries and three user classes: 30% hotspot pedestrians initialised within 400 m of the area centre, 40% uniform pedestrians ($\bar{v} \approx 1.3$ m/s), 30% vehicular ($\bar{v} \approx 12$ m/s). Demand is a clipped-lognormal base modulated by an AR(1) process plus Poisson-triggered $2.4\times$ bursts of 3–6 s. Mean utilisation is $\bar{\rho} = 0.65$ at the nominal load, with persistent spatial skew at the hotspot. Each run is 120 s with the first 10 s discarded as warm-up. Ten evaluation seeds $(1 \ldots 10)$ give paired traces replayed identically across all policies and all Δ; three disjoint seeds $(101 \ldots 103)$ are used for all baseline hyper-parameter tuning.

Baselines. Static: capacity-aware one-shot placement at admission, never migrates (all policies share this initial placement). Follow: migrate to the site co-located with the (lagged) serving cell, i.e. Follow-Me-Edge [5], capacity-blind. RG (reactive greedy): Alg. 1 with the commit test replaced by $\hat{g}_u > 0.1$ ms — an unconditional re-optimiser, with the same incremental-load and budget logic so that it is not a strawman. FH (fixed hysteresis): commit iff $\hat{g}_u > \theta$, with θ grid-searched over $\{0.5, 1, 2, 3, 5, 8, 12\}$ ms on the held-out seeds. We report both FH-fix ($\theta^\star{=}2$ ms, tuned once at the nominal point, as an operator would) and FH-opt ($\theta^\star$ re-tuned at every operating point, an optimistic oracle baseline). Oracle: SHARP with $\Delta = 0$ and noiseless telemetry — not an optimum, but an upper bound that isolates the cost of lag from the cost of the rule.

Metrics. SLO violation rate J (%, migration downtime counted as violation), migrations per user-minute, 95th-percentile latency, and ping-pong ratio (share of migrations returning a user to a site it left within 2 s). We report mean ± 95% CI over 10 seeds ($t_{0.975,9} = 2.262$) and paired t-statistics for SHARP vs. FH.

Reproducibility. Python 3.12.3, NumPy 2.4.4, single vCPU (Intel Xeon @ 2.10 GHz), no GPU, no external data. The full campaign (E1–E3, 500+ runs) completes in 272 s wall-clock. All randomness derives from `numpy.random.default_rng(seed)` with the seeds listed above.

VI. Results and Analysis

F1: lag dominates policy. In Table II the best policy's violation rate rises from 0.91% at $\Delta = 0$ to 10.71% at $\Delta = 6.4$ s — a factor of 11.8. At any fixed Δ, the spread between the best and worst *load-aware* policy is at most

TABLE II

CONTROL-LOOP LAG SWEEP (10 SEEDS, MEAN ± 95% CI). LEFT: SLO VIOLATION RATE (%). RIGHT: MIGRATIONS PER USER-MINUTE.

Δ	violation rate (%)			migrations / user-min		
(ms)	RG	FH-fix	SHARP	RG	FH	SHARP
0	2.37±.07	**0.91**±.05	0.98±.08	13.41	3.49	1.49
200	2.73±.12	**1.29**±.09	1.54±.11	13.39	3.49	1.54
400	3.15±.11	**1.71**±.12	2.13±.11	13.48	3.48	1.57
800	3.99±.20	**2.62**±.22	3.13±.33	13.29	3.47	1.63
1600	5.95±.36	**4.56**±.44	5.11±.47	13.27	3.45	1.69
3200	10.03±.67	**8.50**±.72	8.52±.83	13.09	3.40	1.84
6400	12.67±.84	10.71±.83	**10.23**±.87	12.75	3.32	2.09
Δ-independent references:						
STATIC 7.76±1.00%, 0 mig., p_{95}=47.9 ms ORACLE 0.78±0.06%, 1.77 mig.						
FOLLOW 53.21±2.27%, 15.26 mig., 80.5% ping-pong, p_{95} saturated						

TABLE III

ABLATION: VIOLATION % / MIGRATIONS PER USER-MIN (10 SEEDS)

	Δ=400 ms	Δ=1600 ms	Δ=3200 ms
SHARP (full)	2.13±.11 / 1.57	5.11±.47 / 1.69	8.52±.83 / 1.84
−dead reckoning	1.77±.12 / 1.88	4.98±.44 / 2.10	8.59±.63 / 2.25
−risk calibration	1.80±.15 / 2.71	4.73±.40 / 2.79	8.37±.80 / 2.90

TABLE IV

OPERATING-POINT ROBUSTNESS: VIOLATION % (MIGRATIONS PER USER-MIN). REGRET IS MEASURED AGAINST FH-OPT, THE PER-POINT ORACLE-TUNED THRESHOLD.

Operating point	RG	FH-fix	FH-opt	SHARP
OP1 nominal	3.15 (13.5)	1.71 (3.5)	**1.71** (3.5)	2.13 (1.6)
OP2 Δ=3.2 s	10.03 (13.1)	8.50 (3.4)	**8.50** (3.4)	8.52 (1.8)
OP3 $\bar{\rho}$=0.52	2.28 (14.8)	0.30 (1.7)	**0.17** (0.4)	0.37 (0.2)
OP4 $\bar{\rho}$=0.81	22.57 (11.6)	**22.10** (6.9)	22.18 (8.7)	22.86 (6.0)
OP5 T_{mig}=0.6 s	7.85 (7.1)	4.40 (3.1)	**3.23** (0.9)	3.53 (1.0)
OP6 T_{mig}=1 s, Δ=1.6 s	12.02 (4.6)	9.05 (2.8)	**6.72** (0.8)	7.19 (0.8)
$\theta^\star$ at this point	—	(2.0 fixed)	2/2/5/1/8/8	none
mean regret (pp)	2.565	0.592	—	**0.348**
max regret (pp)	—	2.330	—	**0.680**
mean migrations	10.8	3.6	2.9	**1.9**

2.44 pp (at 6.4 s) and typically ≈ 1.4 pp. Comparing against ORACLE (0.78%), 63% of SHARP's violations at $\Delta = 400$ ms and 85% at $\Delta = 1600$ ms are attributable to staleness alone rather than to the decision rule. The operational consequence is quantitative: moving FH from a 1.6 s loop to an 800 ms loop saves 1.94 pp, whereas replacing RG with the best rule at 1.6 s saves 1.39 pp; from 3.2 s to 1.6 s the numbers are 3.94 pp versus 1.53 pp. *Shortening the control loop is worth* 1.4–2.6× *more than any algorithm substitution we measured.* An operator choosing between an xApp on the near-RT RIC and a smarter rApp on the non-RT RIC should choose the former.

F2: unconditional re-optimisation is never right. RG is the worst load-aware policy at every Δ, while issuing 8× more migrations than SHARP and 3.9× more than FH, with a 23–24% ping-pong ratio versus 1.4–3.3% for SHARP. Its migrations also crowd the per-site transfer budget, so it converts scarce transfer capacity into downtime. FOLLOW is worse still (53.2%): tracking the user without regard to capacity concentrates the hotspot population on four central sites, and its 80.5% ping-pong ratio shows the vehicular users oscillating across cell boundaries. Locality-only policies are unusable at this skew.

F3: where the baseline wins. A tuned fixed threshold beats SHARP on violation rate at every Δ from 200 to 1600 ms, and the gap is significant, not noise: paired differences (FH−SHARP) are −0.25 pp (t= − 7.0) at 200 ms, −0.42 pp (t=−13.4) at 400 ms, −0.52 pp (t=−6.9) at 800 ms and −0.55 pp (t= − 3.1) at 1600 ms. The difference is not significant at $\Delta = 0$ (t= − 1.94) or 3200 ms (t= − 0.08), and reverses at 6400 ms (+0.48 pp, t=2.48). The mechanism is visible in the migration columns: SHARP issues 1.5–2.1 migrations per user-minute against FH's 3.3–3.5, and in the mid-lag regime the extra churn FH buys is worth its downtime, because a wrong move is cheap (150 ms) and gets corrected two epochs later. SHARP is, at these settings, slightly too conservative.

F4: mobility prediction is the wrong knob. Table III ablates SHARP. Removing dead reckoning (SHARP-NP) *improves* violation rate at $\Delta = 400$ ms (1.77 vs. 2.13) and 1600 ms (4.98 vs. 5.11), and only at 3200 ms does extrapolation become marginally useful (8.52 vs. 8.59, well inside the CI). The reason is dimensional: a 12 m/s vehicle covers 19 m in 1.6 s against a 250 m cell radius, so the serving cell almost never changes within the lag window, while a 25 m location error [2] injects velocity noise that occasionally does flip the predicted cell. What actually goes stale is *load*, which drifts on the burst timescale. This contradicts the emphasis of the mobility-aware placement literature [6] for sub-second loops and supports the decomposition in Sec. III. It also validates the calibration: $\hat{\sigma}_1$ grows monotonically with lag ($0.55, 0.65, 0.78, 1.12, 1.70, 2.50, 2.80$ ms for $\Delta = 0 \ldots 6400$ ms) while $\hat{\nu}$ stays flat at 0.93–1.11 ms·s$^{-1/2}$, i.e. the estimator correctly separates lag-induced error from horizon drift. Replacing (4) with a fixed threshold while keeping prediction (SHARP-FT) lowers violations slightly but costs 57–73% more migrations.

F5: the case for calibration is robustness, not peak performance. Table IV evaluates six operating points spanning a 32× lag range, a 2.4× load range and a 6.7× state-size range, and measures each policy's *regret* against FH-opt, the threshold re-tuned by an oracle at that exact operating point. SHARP's mean regret is 0.35 pp (max 0.68 pp); a threshold tuned once at the nominal point incurs 0.59 pp mean and 2.33 pp max regret; RG incurs 2.57 pp. Crucially, FH's optimal $\theta^\star$ moves from 1 ms to 8 ms across these points — an eightfold swing that an operator cannot know in advance — while SHARP tracks it with no tuning at all and with 36% fewer migrations than FH-opt and 47% fewer than FH-fix. The two operating points where a once-tuned threshold falls apart (OP5, OP6) are exactly those where migrations get expensive, i.e. bulky service state, which is where 6G-era stateful edge services are heading.

F6: a regime where nothing matters. At OP4 ($\bar{\rho} = 0.81$) all four policies fall within 0.8 pp of one another with ±2.1 pp confidence intervals, and even the zero-lag ORACLE only

reaches 18.1%. When aggregate capacity is close to aggregate demand, re-placement cannot manufacture headroom; the differences between orchestration policies are an artefact of the moderately loaded regime. Papers reporting placement gains should state their $\bar{\rho}$, because the same algorithm can look transformative at $\bar{\rho} = 0.52$ (RG 2.28% vs. FH 0.30%, a $7.6\times$ ratio) and irrelevant at $\bar{\rho} = 0.81$.

VII. Limitations

These are properties of this study, not of MEC in general.

(i) Queueing fidelity. Eq. (1) is an M/G/1-PS mean-value formula evaluated per slot; it has no per-job variance and no intra-slot arrival correlation. Realised latencies are therefore smoother than in a real system, which biases $\hat{\sigma}_1$ downward and makes SHARP *more* aggressive than it should be. F3 (our loss to FH) would likely narrow, and F5 widen, under a job-level simulator.

(ii) A global σ. σ_1, ν are estimated over all users jointly. The hotspot sites have visibly heavier residual tails than the quiet corner sites, yet receive the same threshold. This is the most probable cause of the 0.42 pp deficit at OP1: SHARP under-migrates precisely where the action is. A per-site or per-decile calibration is the obvious next step and we have not implemented it.

(iii) The error-growth model. $\sigma^2(h) = \sigma_1^2 + \nu^2 h$ assumes zero-mean Brownian error. The measured $\hat{\nu}$ was nearly invariant to Δ (0.93–1.11), which indicates the model captures queue drift but folds the staleness-induced *bias* into σ_1 rather than modelling it. A bias term $\mu(\Delta)$ would be a first-order correction.

(iv) Migration cost model. Migration is a fixed downtime plus per-site transfer concurrency. We do not model backhaul bandwidth contention between state transfer and user traffic, so the true cost of RG's churn is understated; nor do we model iterative pre-copy [13], under which T_{mig} depends on the dirty-page rate and hence on the very load we are balancing.

(v) Topology and mobility. A regular 4×4 grid with Manhattan hop counts and reflecting-boundary Gauss–Markov mobility is not a street network; vehicular users turn more freely than they would on roads, which if anything *flatters* extrapolation — and extrapolation still lost (F4). Results at $N = 16, U = 200$ may not extend to metro-scale deployments where the candidate set $\mathcal{K}_u$ is a small fraction of $\mathcal{N}$.

(vi) Unmeasured comparisons. We did not compare against MPC or deep-RL controllers. Our claim is bounded accordingly: a calibrated commit rule removes the tuning burden at ≈ 0.35 pp cost, not that it is the best achievable policy. We also never estimate Δ itself, and we assume the action log is exact — an orchestrator with failed or partially-applied actuations would face a strictly harder problem.

(vii) Single-machine simulation. No testbed, no real service state, no real RIC. The absolute violation rates should be read as relative, not predictive.

VIII. Conclusion

We showed that for MEC service re-placement the control-loop lag Δ is a first-order design variable: it moves the SLO violation rate by $11.8\times$ across a plausible range, while the choice among load-aware policies moves it by at most 2.4 pp. Within that constraint, we proposed a commit rule that calibrates its migration hysteresis from its own prediction residuals. It does not beat a per-operating-point-tuned threshold — it loses by 0.42 pp at the nominal point — but it stays within 0.68 pp of that oracle across a wide range of lag, load and state size without any tuning, at 36% fewer migrations, and it wins outright once migrations get expensive or the loop gets long. We also report that mobility extrapolation, the field's default remedy for staleness, does not help below multi-second lags. The next step is per-site calibration and an explicit staleness-bias term.

References

[1] ETSI, "Multi-access Edge Computing (MEC); Framework and Reference Architecture," GS MEC 003, 2022.

[2] ETSI, "Multi-access Edge Computing (MEC); Location API," GS MEC 013.

[3] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 2, pp. 1376–1411, 2023.

[4] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Commun. ACM*, vol. 59, no. 5, pp. 50–57, 2016.

[5] T. Taleb and A. Ksentini, "Follow me cloud: Interworking federated clouds and distributed mobile networks," *IEEE Network*, vol. 27, no. 5, pp. 12–19, 2013.

[6] T. Ouyang, Z. Zhou, and X. Chen, "Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 10, pp. 2333–2345, 2018.

[7] S. Wang, R. Urgaonkar, M. Zafer, T. He, K. Chan, and K. K. Leung, "Dynamic service migration in mobile edge computing based on Markov decision process," *IEEE/ACM Trans. Netw.*, vol. 27, no. 3, pp. 1272–1288, 2019.

[8] M. J. Neely, *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan & Claypool, 2010.

[9] M. Mitzenmacher, "How useful is old information?," *IEEE Trans. Parallel Distrib. Syst.*, vol. 11, no. 1, pp. 6–20, 2000.

[10] M. Mitzenmacher, "The power of two choices in randomized load balancing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 12, no. 10, pp. 1094–1104, 2001.

[11] T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta, and D. Sabella, "On multi-access edge computing: A survey of the emerging 5G network edge cloud architecture and orchestration," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1657–1681, 2017.

[12] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 2017.

[13] C. Clark *et al.*, "Live migration of virtual machines," in *Proc. USENIX NSDI*, 2005.

[14] A. Machen, S. Wang, K. K. Leung, B. J. Ko, and T. Salonidis, "Live service migration in mobile edge clouds," *IEEE Wireless Commun.*, vol. 25, no. 1, pp. 140–147, 2018.

[15] T. Camp, J. Boleng, and V. Davies, "A survey of mobility models for ad hoc network research," *Wireless Commun. Mobile Comput.*, vol. 2, no. 5, pp. 483–502, 2002.

[16] P. R. Kalata, "The tracking index: A generalized parameter for α-β and α-β-γ target trackers," *IEEE Trans. Aerosp. Electron. Syst.*, vol. AES-20, no. 2, pp. 174–182, 1984.

[17] W. Saad, M. Bennis, and M. Chen, "A vision of 6G wireless systems: Applications, trends, technologies, and open research problems," *IEEE Network*, vol. 34, no. 3, pp. 134–142, 2020.

Appendix: Disclosure Statement

Authorship. This paper was produced end-to-end by Claude Opus 5, a large language model developed by Anthropic,

acting as an autonomous researcher in a single session. Problem selection, system model, algorithm design, simulator implementation, experiment design, execution, analysis, and all prose are the model's output. No human wrote, edited, or reviewed any sentence of this paper or any line of the simulator prior to submission. Degree and nature of human oversight of the session: [PLACEHOLDER]. Institutional affiliation under which this submission is made: [PLACEHOLDER].

Computation. All numerical results were obtained by executing the simulator described in Sec. III–V inside a sandboxed Linux container: Python 3.12.3, NumPy 2.4.4, one vCPU (Intel Xeon @ 2.10 GHz), no network access during execution, no GPU. The complete campaign (lag sweep, ablation, load sweep, operating-point study, baseline tuning; 500+ runs) took 272 s wall-clock. Whether that container was co-resident with other workloads, and thus whether the reported wall-clock timings are contended, is unknown to the author: [PLACEHOLDER]. No result in this paper is analytical, interpolated, or reproduced from a third-party source; every figure quoted in the text appears in an executed run's output. Simulator source and raw logs: [PLACEHOLDER] (repository URL).

Bibliographic verification. The references were selected from the model's parametric knowledge. During this session the model did not query a live bibliographic database, so volume, issue, and page numbers may contain errors even where the work itself exists. Reviewers should treat the reference list as unverified metadata attached to correctly identified works, and the author of record accepts responsibility for correcting it: [PLACEHOLDER].

Conflict of interest in the evaluation. The same agent designed both the proposed method and the evaluation that judges it, which is a structural conflict. Three mitigations were applied and are auditable in the code: (a) all baseline hyper-parameters were tuned by grid search on seeds disjoint from the evaluation seeds; (b) an oracle-tuned variant of the strongest baseline (FH-opt) was included, which the proposed method does not beat; (c) results where a baseline wins (F3), where the proposed method's own components fail (F4), and where no method separates (F6) are reported in the main results section rather than deferred. No experiment was run and discarded after seeing its outcome except the parameter-calibration runs described in Sec. V (queue and load scaling to place the system in a non-degenerate regime), which were performed before any policy comparison was made.

Funding, competing interests, prior/concurrent submission, and ethics review. [PLACEHOLDER].